



Классификация задач по этапам жизненного цикла ПО

Сегодня мы будем рассматривать искусственный интеллект не как отдельную технологию, которую нужно обязательно встроить во все процессы разработки, а как инструмент, который может применяться на разных этапах жизненного цикла программного обеспечения. Важно сразу понимать, что задачи на разных этапах сильно отличаются. Поэтому и польза от ИИ будет разной.

Жизненный цикл программного обеспечения — это весь путь программного продукта от первоначальной идеи и проектирования до эксплуатации и дальнейшего сопровождения. В нашем случае мы сосредоточимся на шести этапах: проектировании, кодировании, тестировании, ревью, деплое и сопровождении.

Здесь важно не воспринимать эти этапы как абсолютно изолированные процессы. В реальной разработке команда может возвращаться к проектированию после обнаружения проблемы во время тестирования, изменять код после ревью или выпускать исправление уже после развёртывания приложения. Поэтому правильнее рассматривать жизненный цикл как постоянно повторяющийся процесс улучшения программного продукта.

ИИ на этапе проектирования

На этапе проектирования мы ещё не обязательно пишем код. Здесь разработчики, аналитики и архитекторы пытаются понять, что именно необходимо создать и каким образом система должна решать поставленную задачу.

ИИ на этом этапе особенно полезен при работе с большим количеством текстовой информации. Например, мы можем передать описание требований и попросить выявить противоречия, найти неясные формулировки или предложить вопросы, которые необходимо уточнить у заказчика.

Представим, что в требованиях написано: «Пользователь должен быстро получать информацию о заказе». Для человека сразу возникает вопрос: что означает слово «быстро»? Одна секунда, пять секунд или десять секунд? ИИ может обратить внимание на подобные неопределённые формулировки.

Также ИИ может использоваться для предварительной декомпозиции задачи. Если мы описываем новую функциональность, модель может помочь определить, какие части системы потенциально придётся изменить. Но здесь особенно важно помнить, что результат ИИ является предложением, а не готовым архитектурным решением.

На этапе проектирования наиболее полезная роль ИИ заключается в ускорении анализа информации и подготовке вариантов для дальнейшего рассмотрения человеком.

ИИ на этапе кодирования

На этапе кодирования область применения становится значительно шире, потому что модель уже работает непосредственно с программным кодом.

Один из наиболее очевидных сценариев — генерация небольших фрагментов программы. Разработчик может описать требуемое поведение функции и получить начальный вариант реализации. Это особенно удобно при работе с повторяющимися конструкциями, типовыми обработчиками, преобразованием данных или стандартными операциями.

Другой распространённый сценарий — автодополнение. Разработчик начинает писать функцию, а система предлагает продолжение на основании уже написанного кода и окружающего контекста.

ИИ может помогать и при изменении существующего кода. Например, разработчик может передать функцию и попросить сделать её более читаемой, удалить повторяющиеся участки или адаптировать под другую версию библиотеки.

Но скорость генерации кода сама по себе ещё не означает увеличение производительности. Если разработчик получил код за десять секунд, а затем двадцать минут исправлял ошибки, реального выигрыша может не быть. Поэтому позже нам обязательно понадобится оценивать не количество сгенерированного кода, а его практическую пригодность.

ИИ на этапе тестирования

На этапе тестирования задача меняется. Здесь нас интересует уже не создание функциональности, а проверка её поведения.

ИИ может помочь сформировать варианты тестовых сценариев на основании требований или кода. Например, если функция принимает возраст пользователя, модель может предложить проверить стандартное значение, минимально допустимое значение, максимально допустимое значение, отрицательное число или некорректный тип данных.

Это особенно полезно потому, что разработчик часто концентрируется на основном сценарии работы программы. ИИ может помочь обратить внимание на граничные и нестандартные случаи.

Также модель может анализировать сообщение об ошибке, результаты выполнения тестов или описание обнаруженного дефекта и помогать сформировать возможные причины проблемы.

Но ИИ не заменяет сам механизм тестирования. Если модель написала тест, этот тест всё равно необходимо выполнить. Только фактический запуск программы показывает, проходит проверка или нет.

Поэтому на этапе тестирования ИИ прежде всего помогает быстрее создавать проверочные сценарии, расширять набор рассматриваемых случаев и анализировать результаты.

ИИ на этапе ревью

Ревью кода означает проверку изменений перед тем, как они станут частью основной версии проекта.

Обычно разработчик создаёт изменение, после чего другой участник команды изучает код. Он проверяет корректность логики, читаемость, соответствие правилам проекта и возможные проблемы.

ИИ может выполнять предварительный анализ изменений и обращать внимание разработчика на подозрительные участки. Например, он может заметить повторяющийся код, слишком сложное условие, потенциально отсутствующую обработку ошибки или несогласованность между несколькими частями программы.

Здесь особенно важно различать подсказку и подтверждённую проблему. Если ИИ написал: «В этом месте возможна ошибка», это ещё не означает, что ошибка действительно существует. Такое замечание необходимо проверить.

Следовательно, ценность ИИ в ревью заключается не в том, что он автоматически объявляет код правильным или неправильным, а в том, что он помогает быстрее находить места, которые заслуживают дополнительного внимания.

ИИ на этапе деплоя

Когда разработка и проверка завершены, приложение необходимо развернуть в рабочем окружении. Этот этап называется деплоем.

Здесь ИИ может использоваться для анализа конфигураций, объяснения ошибок развёртывания и подготовки вспомогательных команд или конфигурационных фрагментов.

Например, процесс развёртывания завершился ошибкой. Вместо того чтобы вручную изучать длинное сообщение, разработчик может использовать ИИ для первоначальной расшифровки ошибки и определения того, на какой части процесса стоит сосредоточиться.

ИИ также может помочь сравнить две версии конфигурации и объяснить, какие изменения потенциально могли повлиять на результат.

Но любые действия, связанные с рабочей инфраструктурой, требуют особенно осторожного подхода. Ошибка в обычной функции может затронуть один компонент программы, а ошибка при развёртывании способна сделать недоступным весь сервис.

Поэтому на этом этапе ИИ разумно воспринимать прежде всего как средство анализа и подготовки решения, а не как источник команд, которые нужно выполнять без проверки.

ИИ на этапе сопровождения

После выпуска программного продукта работа над ним не заканчивается. Начинается этап сопровождения.

Пользователи обнаруживают новые проблемы, появляются изменения требований, обновляются зависимости, накапливаются технические задачи. Кроме того, команда постоянно получает журналы работы приложения, сообщения об ошибках и обращения пользователей.

ИИ здесь может помогать анализировать большие объёмы диагностической информации. Например, разработчик может передать сообщение об ошибке вместе с фрагментом журнала и попросить объяснить наиболее вероятное место возникновения проблемы.

Он также может использовать ИИ для объяснения старого кода. Это особенно актуально в проектах, которые существуют несколько лет и содержат участки, написанные другими разработчиками.

Получается, что на этапе сопровождения одна из основных задач ИИ — сократить время между появлением проблемы и пониманием того, где её необходимо искать.

Почему важно классифицировать задачи

Теперь возникает важный вопрос: зачем вообще разделять применение ИИ по этапам жизненного цикла?

Потому что формулировка «мы используем ИИ в разработке» практически ничего не говорит об эффективности. Один разработчик может использовать его только для объяснения ошибок, другой — для генерации функций, третий — для создания тестов. Это совершенно разные сценарии.

Если мы хотим оценить пользу ИИ, сначала необходимо определить конкретную задачу. Например, не просто «используем ИИ при тестировании», а «используем ИИ для подготовки дополнительных тестовых сценариев».

После этого уже можно сравнивать процесс до использования ИИ и после его появления.

Именно здесь появляются метрики эффективности.

Метрика Time-to-first-response

Первая метрика, которую мы рассмотрим, называется time-to-first-response. В нашем контексте это время от момента, когда разработчик сформулировал запрос или задачу для ИИ, до момента получения первого полезного ответа.

Ключевое слово здесь — полезного.

Если система ответила через две секунды, но результат совершенно не связан с задачей, формально ответ пришёл быстро, но с точки зрения рабочего процесса это ничего нам не дало.

Представим ситуацию. Разработчик столкнулся с ошибкой компиляции. Без ИИ он начинает искать документацию, открывает несколько страниц, изучает похожие случаи и через десять минут понимает причину. С ИИ он передаёт текст ошибки и через двадцать секунд получает объяснение, которое позволяет сразу продолжить работу.

В таком случае time-to-first-response показывает, насколько быстро разработчик получил отправную точку для решения проблемы.

Эта метрика особенно полезна для задач, где раньше значительное время уходило именно на поиск информации или первоначальное понимание проблемы.

Доля принятых подсказок

Следующая метрика показывает, какая часть предложений ИИ реально используется человеком.

Предположим, система за определённый период предложила сто вариантов продолжения кода. Разработчик принял двадцать из них. Тогда доля принятых подсказок составит двадцать процентов.

Формально расчёт можно представить так:

$$\text{доля принятых подсказок} = \frac{\text{количество принятых подсказок}}{\text{количество всех предложенных подсказок}} \times 100 \%$$

Но интерпретировать такую метрику нужно осторожно.

Очень высокая доля принятия не всегда означает высокое качество системы. Разработчик может автоматически принимать предложения, практически не проверяя их. И наоборот, низкая доля не обязательно говорит о плохой работе ИИ. Возможно, разработчик использует его только для сложных нестандартных задач.

Поэтому эта метрика показывает прежде всего практическую востребованность предложений ИИ в конкретном рабочем процессе.

Если большая часть подсказок постоянно отклоняется, это сигнал, что система плохо соответствует текущим задачам или выдаёт слишком много нерелевантных предложений.

Количество исправленных дефектов

Третья метрика связана непосредственно с качеством программного продукта — количеством дефектов, которые удалось обнаружить или исправить при использовании ИИ.

Допустим, во время анализа кода ИИ указал на потенциальную проблему с обработкой пустого значения. Разработчик проверил этот участок, подтвердил ошибку и исправил её до выпуска новой версии. Такой случай уже можно учитывать как реальный исправленный дефект.

Здесь принципиально важно считать именно подтверждённые проблемы.

Если ИИ написал десять замечаний, но восемь оказались ошибочными предположениями, нельзя говорить, что он обнаружил десять дефектов. Фактически полезными оказались только два подтверждённых замечания.

Эта метрика хорошо показывает разницу между активностью ИИ и реальным результатом. Система может генерировать огромное количество текста, комментариев и рекомендаций, но ценность появляется только тогда, когда это приводит к конкретному улучшению программного продукта.

Как правильно оценивать эффективность ИИ

Мы не должны делать вывод об эффективности ИИ только по одной цифре.

Представим, что время получения ответа сократилось с десяти минут до десяти секунд. На первый взгляд результат отличный. Но если разработчик затем вынужден пятнадцать минут исправлять полученный код, общая производительность могла даже снизиться.

Или другая ситуация. Доля принятых подсказок составляет всего пятнадцать процентов. Это может показаться плохим показателем. Но если эти пятнадцать процентов помогают разработчику решать наиболее сложные задачи и экономят несколько часов работы, инструмент всё равно может быть полезен.

Поэтому эффективность ИИ всегда нужно связывать с конкретной задачей и конкретным этапом жизненного цикла.

Для задач, связанных с быстрым поиском и объяснением информации, особенно интересно время получения полезного ответа. Для автодополнения кода важной становится доля реально принятых предложений. Для анализа качества кода и тестирования можно дополнительно посмотреть, сколько реальных дефектов удалось обнаружить и исправить.

Основной вывод

На этом этапе я хочу, чтобы мы зафиксировали одну идею: ИИ в разработке — это не одна задача и не один сценарий использования.

На проектировании он помогает анализировать требования и варианты решения. Во время кодирования он ускоряет создание и изменение программного кода. При тестировании помогает формировать проверки и анализировать ошибки. На ревью может выполнять предварительный анализ изменений. Во время деплоя помогает разбираться с конфигурациями и ошибками развёртывания. На сопровождении ускоряет поиск причин неисправностей и работу с существующим кодом.

При этом сам факт использования ИИ ещё не означает повышения эффективности. Нам необходимо измерять результат. Именно поэтому появляются такие показатели, как время до первого полезного ответа, доля принятых подсказок и количество реально исправленных дефектов.

И в дальнейшем, когда мы будем рассматривать конкретные способы применения ИИ, нам будет важно постоянно задавать один и тот же вопрос: не просто «может ли ИИ выполнить эту задачу?», а «даёт ли использование ИИ измеримое преимущество в этой задаче?».
